PENSE-BÊTE · TECHNIQUE ET COMMANDES

Gestion des processus Linux-Unix

09-190 PSE-ER F2-7 Linux-Unix - gestion des processus — source : fiche de révision du module (gemma4, 03/11/2026 ; relu par Claude le 04/11/2026)

À RETENIR

1. Un programme est un objet statique tandis qu'un processus est un objet dynamique constitué d'un programme en exécution et de son contexte.
2. Le mécanisme fork duplique un processus et retourne le PID du fils dans le père, 0 dans le fils, ou -1 en cas d'échec.
3. Les threads partagent l'essentiel des ressources avec le processus sauf le compteur ordinal, la pile et le jeu de registres.

Généralités sur les processus

Notion	À retenir
Programme et processus	Programme : suite d'instructions, objet **statique** dans un fichier. Processus : déroulement de l'exécution, objet **dynamique** (programme + contexte).
Le changement de contexte	Sauvegarde le contexte d'un processus et rétablit celui du suivant. Survient après chaque **quantum**, quand le noyau prend la main ou lors d'une **entrée-sortie**.
L'ordonnancement	Ordre d'attribution du processeur pour un **quantum**. Critères : **efficacité**, **rendement**, **temps d'exécution**, **interactivité** et **équité**.
Le pseudo-parallélisme	Commutation rapide par le **scheduler** simulant le parallélisme. La **réentrance** permet d'exécuter plusieurs instances d'un même programme.

Hiérarchie et cycle de vie des processus

Notion	À retenir
Père et fils	Filiation en arbre : chaque fils a **un seul père**. Au démarrage, `init` est l'ancêtre unique. `init` crée des processus **interactifs** (terminal) ou **daemons** (aucun terminal).
Le mécanisme fork et exec	`fork` duplique (copy on write) et retourne le **PID du fils** (au père), **0** (au fils) ou **-1** (échec). `exec` écrase le programme (PID conservé). `wait` attend le fils. Après `exit`, le processus reste **zombie** tant que le père n'a pas lu son code de sortie.
Les threads	Création **60 pour cent** plus rapide que `fork`. Partagent tout sauf **compteur ordinal, pile et registres**. Risques : **race conditions**, **non-réentrance** (doit être **thread safe**) et **deadlocks**.

Typologie et attributs des processus

Notion	À retenir
Les catégories de processus	Processus interactifs (terminal, avant/arrière-plan) · processus en lots (sans terminal, file d'attente) · démons/serveurs (permanents, via scripts d'initialisation comme `crond` ou `syslogd`)
Les attributs	Identifiants : `PID` (conservé à l'**exec**), `PPID` (père), `UID` (réel, effectif, sauvegardé), `GID`, `EGID`, `SGID` · Autres : `tty`, `pri`, `s`, `stime`, `time`, `size`

Modes d'exécution et gestion

Notion	À retenir
Modes d'exécution	Synchrone : fils en **avant-plan**, exécution en **séquence** ; Asynchrone : fils en **tâche de fond**, exécution en **parallèle**.
Entrées-sorties	Descripteurs **0** (entrée), **1** (sortie), **2** (erreur). `>` écrase, `>>` ajoute à la fin, `<` donne l'entrée, `2>` redirige l'erreur, `&>` sortie + erreur ; périphériques = fichiers de `/dev`.
Surveillance	`ps` (`ps -aux` BSD, `ps -edf` AT&T), `kill` (signal à un processus du même propriétaire), `kill -l`, `limit`. Signaux : **SIGINT** (Ctrl-C), **SIGKILL** (tue), **SIGTERM** (fin propre), **SIGHUP** (relire la configuration), **SIGCHLD** (fin d'un fils), **SIGSEGV**.

Planification des processus

Notion	À retenir
La commande at	Lance un processus de façon différée avec un paramètre de date ; `at -l` liste les processus en attente ; **Ctrl-D** sort de la zone de commande.
La crontab	Lance des commandes à intervalles réguliers ou dates ; édition via `-e` (éditeur par défaut) ; liste via `-l` ; reconnaît `USER`, `HOME` et `SHELL`.

Gestion des packages

Notion	À retenir
Principe et HP-UX	Ressources en **filesets** (**Sub Products**), **Products** (ensembles de filesets) et **Bundles** (collections HP). HP-UX 10.20 utilise des **Bundles**. Commandes : `swlist`, `swinstall`, `swremove`.
Synthèse et APT	Paquet = archive compressée avec programmes d'installation et dépendances. **APT** (Debian) automatise la recherche dans des **dépôts** selon la version.

Pièges et confusions

- ⚠ Confondre le programme (objet statique) et le processus (objet dynamique).
- ⚠ Croire que le PID change lors d'un appel exec (il est conservé).
- ⚠ Oublier qu'un processus terminé devient un zombie si le père n'a pas récupéré son code de sortie.
- ⚠ Confondre le chevron simple qui écrase avec le double chevron qui ajoute à la fin.
- ⚠ Inverser les descripteurs 1 et 2 (1 est la sortie standard, 2 est la sortie d'erreur).
- ⚠ Se tromper sur les options de ps entre les systèmes BSD (ps -aux) et AT&T (ps -edf).

Sigles et définitions

Multitâche	Capacité de Linux à gérer plusieurs programmes simultanément.
Quantum	Laps de temps pendant lequel un processus dispose du processeur.
Pseudo-parallélisme	Simulation d'une exécution simultanée par un ordonnanceur qui bascule rapidement d'un processus à l'autre.
Copy on write	Mécanisme ne copiant effectivement le processus qu'au moment où une modification intervient.
Daemon	Processus serveur tournant en permanence, rattaché à aucun terminal.
PID	Nombre identifiant un processus, utilisé par le noyau pour l'administration.
Thread	Sous-processus constitué d'un compteur ordinal, d'une pile et de registres, partageant le reste avec le processus.

Auto-test

1. Quelle est la valeur retournée par fork dans le processus fils ?
2. Quel est l'ancêtre de tous les processus au démarrage ?
3. Quel état prend un processus après un appel exit si le père ne récupère pas son code ?
4. Quel signal est envoyé par la combinaison Ctrl-C ?
5. Quel descripteur correspond à la sortie d'erreur ?

Réponses : **1.** 0 · **2.** init · **3.** zombie · **4.** SIGINT · **5.** 2

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.